

SuperSpec: Enhanced Verification and Sampling for End-to-End LLM Speculative Decoding

Chen Shen
*Department of Intelligence and
Computing
Tianjin University
Tianjin, China*

Rui Guo
*Department of Intelligence and
Computing
Tianjin University
Tianjin, China*

Yang Cheng
*Department of Intelligence and
Computing
Tianjin University
Tianjin, China*

Yang Lin
*Department of Intelligence and
Computing
Tianjin University
Tianjin, China*

Zhixin Zhao
*Department of Intelligence and
Computing
Tianjin University
Tianjin, China*

Yitao Hu*
*Department of Intelligence and
Computing
Tianjin University
Tianjin, China
yitao@tju.edu.cn*

Sheng Chen
*Department of Intelligence and
Computing
Tianjin University
Tianjin, China*

Xiulong Liu
*Department of Intelligence and
Computing
Tianjin University
Tianjin, China*

Keqiu Li
*Department of Intelligence and
Computing
Tianjin University
Tianjin, China*

Abstract—Modern LLM decoding has the drawbacks of high cost and slow speed, and speculative decoding has been shown to be an effective solution to this problem. However, the inference latency still poses a significant challenge to maintaining service level objectives (SLOs) in systems that employ multiple draft models for speculative decoding. The verification phase in such systems if reliant on tree attention often constitutes a bottleneck especially when draft sequences lack common prefixes and substantially underutilizes GPU parallelism while increasing end-to-end latency. We introduce SuperSpec, an end-to-end speculative decoding system designed to co-optimize verification, sampling and draft generation. SuperSpec integrates three pivotal innovations: an *Efficient Batch Verifier*, which substitutes tree-based flattening with batch parallel validation and layer-wise KV Cache replication; a *Global Optimal Sampler*, which assesses all candidate sequences within a batch to ascertain the longest valid path, thereby circumventing the local optima frequently encountered in tree-based rejection sampling; and a *Dynamic Adaptive Multi-Drafter*, which dynamically modulates the speculative length (K) for each drafter predicated on real-time idleness metrics and acceptance rates. Empirical evaluations of Qwen2.5-72B and the OPT-66B on various datasets show that SuperSpec improves average acceptance rate by 6.4% to 30.2%, and the end-to-end inference acceleration ratio by 7.12% to 62.06%, when compared to the state-of-the-art tree-based speculative decoding system SpecInfer. These improvements were achieved without compromising the quality of text generation, making SuperSpec an effective solution for accelerating LLM inference.

This work was supported in part by the National Key Research and Development Program of China under Grant 2022YFB4501000, and in part by the National Natural Science Foundation of China under Grant 62202328.

Index Terms—Distributed LLM Inference; Speculative Decoding; Parallel Verification; Global Optimal Sampler

I. INTRODUCTION

Large Language Models (LLMs) have demonstrated unprecedented capabilities in a variety of application areas, from real-time dialogue systems[1] and instant code completion[2] to intelligent customer service[3]. In these scenarios, compliance with Service Level Objectives (SLOs), which are customarily operationalized via Time to First Token (TTFT) and Time Per Output Token (TPOT), is critical to preserve user-perceived responsiveness. For instance, live-stream translation systems require TPOT under 30 ms per token while processing individual requests to avoid conversational lag. Speculative decoding has demonstrated effective acceleration of LLM inference by performing draft generation and verification in parallel. However, the verification phase constitutes a latency bottleneck in scenarios employing multiple draft models simultaneously that substantially increases the overall end-to-end decoding latency.

Modern speculative decoding systems face a critical tension: while increasing speculative length (K) improves throughput by amortizing verification costs, it risks violating TPOT constraints due to verification phase bottlenecks. For example, our analysis of the Qwen2.5-14B model in Fig.2 reveals that the verification phase contributes over 60% of the total latency across datasets like Alpaca[4] and GSM8K[5]. As the number of model parameters increases, the verification phase

constitutes a progressively larger proportion of the overall inference latency. This inefficiency stems from the prevailing tree-based attention mechanism[6–8], which organizes candidate tokens into flattened tree structures for parallel validation. Although tree attention reduces redundant computations by reusing shared prefixes, its effectiveness diminishes when draft sequences lack common prefixes, which is a frequent occurrence in practical settings. For instance, 78% of requests in the Alpaca dataset and 96% in CNN-DailyMail[9] exhibit no shared prefixes as shown in Fig.3, resulting in excessively long verification sequences that underutilize GPU parallelism. Such scenarios exacerbate memory bandwidth contention and idle computational resources, as GPUs optimized for GEMM operations struggle with irregularly long sequences. Consequently, even state-of-the-art multi-drafter systems[10, 11], which enhance draft diversity, inadvertently prolong verification latency and trap sampling in local optima, as illustrated in Fig.4.

To alleviate these challenges, we propose SuperSpec, an end-to-end speculative decoding system that co-optimizes verification, sampling and draft generation. First, SuperSpec introduces an *Efficient Batch Verifier* that replaces tree-based flattening with batch-parallel validation. By grouping short draft sequences into GPU-friendly batches, this approach maximizes computational density while reducing peak memory usage to $1/L$ (where L is the model’s layer count) via layer-wise KV Cache replication. Second, a *Global Optimal Sampler* evaluates all candidate sequences within a batch to select the longest valid path, circumventing the local optima inherent in tree-based rejection sampling. Third, a *Dynamic Adaptive Multi-Drafter* dynamically adjusts K per drafter based on real-time idleness metrics and acceptance, eliminating the “bubble phenomenon” where faster drafters idle while waiting for slower peers. The integration of these techniques collectively enables the SuperSpec system to achieve a reduction in end-to-end inference latency.

Our contributions are summarized as follows:

- SuperSpec is the first inference system to identify that tree-based decoding has become a latency bottleneck during the verification phase and to conduct a systematic analysis. This provides new insights for the optimization of speculative decoding systems.
- SuperSpec designs *Efficient Batch Verifier* to alleviate the latency bottleneck of tree-based verifiers, implements *Batch-Based Sampler* to achieve globally optimal solutions, and introduces the *Bubble Controller* and *Adaptive-K Controller* to enhance decoding efficiency and acceptance rate in the drafting phase, thereby reducing end-to-end inference latency.
- Empirical validation indicates that, compared with the tree-based decoding of state-of-the-art systems, SuperSpec achieves a **6.4% to 30.2%** increase in acceptance rate, and a **7.12%–62.06%** increase in end-to-end inference speedup across three real-world datasets within the Qwen2.5-72B and OPT-66B models.

II. RELATED WORK

Speculative Decoding has become a core technology for breaking the bottleneck of latency and throughput in LLM inference through the synergistic mechanism of draft model pre-generation and target model verification. In recent years, research has focused on two aspects: improving acceptance rate and improving performance in high throughput scenarios.

A. Heterogeneous Draft Generation for High Acceptance Rate

Research in this direction focuses on how to improve the acceptance rate by generating or optimizing draft tokens in a variety of ways. SmartSpec[12] defines Goodput metrics, which are combined with system history information to dynamically determine the length of the speculation. Sequoia[13] uses probabilistic concatenated products to model the acceptance rate of nodes, and combines it with dynamic programming to solve the optimal tree structure. Opt Tree[14] uses adaptive token tree design to prune the token tree. In order to better reuse historical token information and improve computational efficiency, Token Recycling[15] uses historical candidate tokens to build a dynamic retrieval library, based on which Ouroboros[16] proposes phrase-level draft generation, storing and verifying the reusable phrases of discarded drafts and generating multiple phrase chunks in parallel. Lookahead Decoding[17] uses the residual arithmetic power to parallelly generate and verify n grams to accelerate. Taking different approaches to larger-scale knowledge reuse, LLMA[18] achieves lossless acceleration using reference text matching, while Rest[19] relies on domain knowledge base retrieval to improve generation quality. Additionally, some other works improve draft generation quality through fine-tuning. Kangaroo[20] reduces parameters by forming a draft model from an LLM’s initial layers and a trained adaptor; Medusa[8] employs a parallel MLP prediction header; and Eagle[11] focuses on resolving uncertainty through feature-level auto-regression and fusing the previously sampled token’s embedding with current features. Notably, these latter methods all invariably utilize tree attention for verification.

B. System Unification Optimisation under High Request Load

Some studies start with speculative end-to-end system optimization with the aim of improving the inference system throughput of LLM under high request service load. PEARL[21] reduces the problem of mutual waiting between the draft model and the target model by continuing to generate drafts through pre-verification early termination and post-verification strategies. PipeInfer[22] reduces the problem through asynchronous speculation, where draft token inference and speculative tree verification can be done in a separate computation flow. Minions[23] uses pipelined execution mechanism to decouple SSM decoding and LLM verification and achieves computational overlap through mini-batch scheduling. CoSine[24] further develops dynamic routing algorithms to support multi-node confidence convergence and load balancing, dynamically allocating the most appropriate resources for each stage. Speculative Streaming[25] enables future n -gram

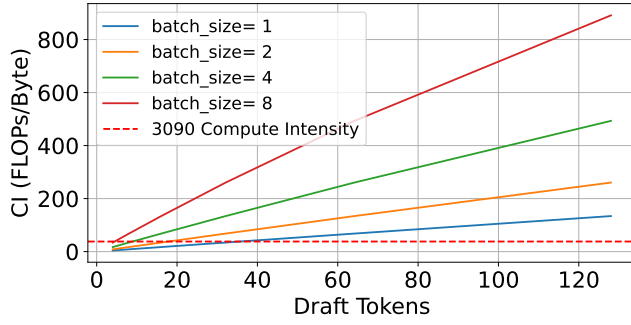


Fig. 1: *CI* Trends for Qwen2.5-14B Model

prediction of the target model with multi-stream attention, while generating and validating drafts. However, all of these efforts invariably use tree attention for verification.

III. BACKGROUND

A. Lifecycle of Speculative Decoding

LLMs Decoding usually requires the use of autoregressive sampling. Its inference process is quite slow and requires serial decoding token by token. Speculative decoding[26, 27] uses an approximate small model to autoregressively decode k steps first, and then sends the results together into the LLM for one forward propagation validation, so as to achieve the effect that one step of decoding in the LLM yields $\alpha \times k$, where α is the acceptance rate. The earliest speculative decoding research used multiple decode heads to generate multiple tokens [28]. Later on, scholars have also proposed the use of historical token matching[17], fine-tuned multi-head[8] and enhanced knowledge base retrieval[19] to obtain draft tokens.

Speculative decoding can be divided into two phases in terms of lifecycle: draft-verify. As shown in (1), we used mathematical analysis to derive end-to-end acceleration ratios for speculative decoding strategies.

$$\text{Speedup} = \frac{(k \times \alpha + 1) \times T_{\text{auto}}}{k \times T_{\text{draft}} + T_{\text{verify}}} \quad (1)$$

The performance of speculative decoding is influenced by four key factors: draft latency (T_{draft}), lookahead (k), which determines how many tokens are drafted in advance, the acceptance rate (α), representing the proportion of draft tokens accepted by LLM and verify latency(T_{verify}). Furthermore, T_{auto} represents the autoregressive decoding latency. According to 1, achieving a higher acceptance rate (α), a lower draft latency (T_{draft}) and a lower verify latency(T_{verify}) leads to better acceleration.

B. Tree Attention Verification for Multi-Drafter Decoding

Early studies hypothesized that the collaborative decoding between large and small models employed a single small-large model architecture[26, 27]. To improve the acceptance rate of LLM, multiple candidate sequences are generated in parallel, merged into a token tree and evaluated by LLM[6], which

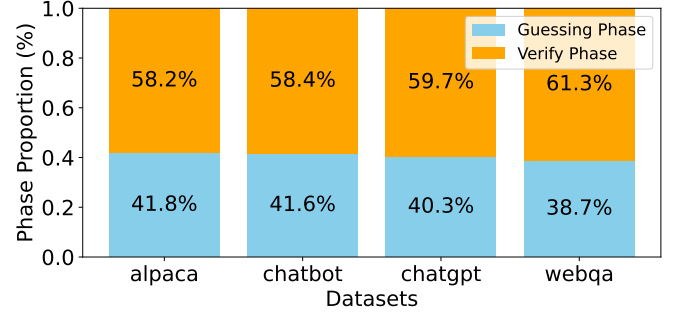


Fig. 2: Latency Percentage in Draft and Verify Phases

selects the most suitable continuation based on a predefined criterion.

Traditional attention is designed for sequential processing, which restricts each token to focus only on the preceding tokens. To support parallel verification of multiple candidate sequences, tree attention merges the candidate sequences into a tree structure and uses a topology-aware causal masking matrix for verification, so that each node in the token tree can only see all the tokens from the root node of the tree to that node, and receives no interference from other sequences[6, 8]. This approach computes all token concerns in a single kernel, thus enabling parallel verification of multiple paths in a single forward pass.

IV. MOTIVATION

In this section, we derive three motivations to guide the design of SuperSpec by performing the analysis of the diverse tasks on models with different parametric quantities in the same family.

Motivation #1: There is an overlooked bottleneck in the validation phase that leads to its higher latency share. The inference pipeline of large language models has widely adopted the speculative decoding framework, which consists of two fundamental components: the drafting phase and the verification phase[29]. Most of the previous studies follow the optimisation idea of increasing the acceptance rate of speculative decoding or reducing the cost of acquiring draft token, but these approaches ignore the space for optimisation in the validation phase. We use the roofline model[30] to analyse the change in computational intensity(*CI*) in the verification phase. (*CI*) quantifies floating-point operations per memory access unit. High *CI* indicates compute-bound tasks, while low *CI* reflects memory-bound constraints. We analyze the *CI* of the Qwen2.5-14B-Instruct model in detail, including the computation and accesses of the attention and ffn modules, as well as the KV Cache, the inputs and the accesses of the outputs, to obtain Fig.1:

As shown in Fig.1, as the number of candidate sequence tokens increases, the bottleneck of the parallel verification process of the Qwen2.5-14B-Instruct model shifts from the access bottleneck to the computational bottleneck, resulting in a significant increase in the verification phase.

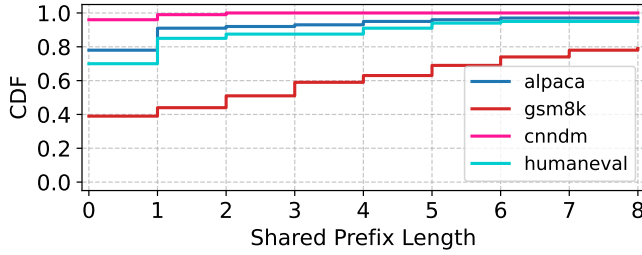


Fig. 3: Shared Prefix Length of Different Datasets

We performed latency statistics for both draft and verify phases of speculative decoding on four real scenario open source datasets alpaca[4], chatbot[31], chatgpt[32] and webqa[33] for the Qwen2.5-14B-Instruct[34] model. The experiments obtain draft tokens from three mini-models Qwen2.5-0.5B, Qwen2.5-1.5B and Qwen2.5-3B, and the verification phase uses the widely used tree attention verification mechanism. The experimental results in Fig.2 show that the delay share of the validation phase is always about 60% of the total delay on different datasets, thus making the validation phase a major bottleneck for performance improvement and potential for optimisation exists.

Motivation #2: Multi-draft model speculative decoding leads to an excessively long sequence of to-be-verified sequences, further aggravating the bottleneck of the tree-attention verification. In order to make speculative decoding achieve a high acceptance rate on multiple types of tasks in practical scenarios, a common approach is to use a multi-draft model to obtain draft tokens to increase the diversity of candidate tokens, so as to increase the probability that the token tree contains the actual output token of the LLM. When using a multi-draft model to obtain draft tokens, the draft sequences generated by each draft model may contain common token prefixes, at this time, if the tree attention mechanism is selected for verification in the verification phase, when the shared prefixes are short and the token tree branches are many, it will cause a serious increase in verification delay and decrease in overall inference speed.

We have analysed the number of shared prefixes when each request is speculatively decoded on four real scenario datasets alpaca[4], gsm8k[5], cnn-dailymail[9] and humaneval[35] also using Qwen2.5 family of models. As shown in Fig. 3, the requests with no shared prefixes account for 78%, 39%, 96% and 70% of the four datasets respectively, and these requests produce longer lengths of sequences to be verified in the verification phase of the tree-attention mechanism. And if we use more miniatures, or augmented retrieval[19], character search[17], etc. to obtain draft token, the length of the pending validation sequence will increase further, which brings the possibility of further increase in decoding delay.

We note that the tree-attention mechanism flattens the sequence for processing, and too long to-be-verified sequences may lead to idle computational resources that cannot efficiently utilise the SM computational cores of the GPU. In addition,

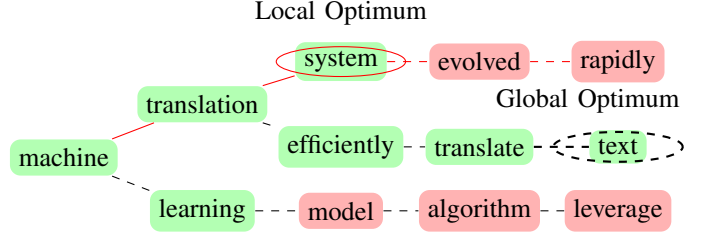


Fig. 4: MSS Verification Algorithm (Stuck in Local Optimum)

due to the natural parallel computing nature of GPUs, which are deeply optimised for GEMM operators, the overhead of a small increase in batch size is much smaller than the gain from a reduction in sequence length. Therefore, there is a great potential for optimisation of the current tree-based attention verification mechanism.

Motivation #3: Multi-draft speculative sampling based on the tree attention mechanism does not guarantee to fetch the globally optimum. The prevailing multi-draft speculative sampling(MSS) relies on a tree-based multi-step verification approach[6]. This strategy while effective in limited contexts often becomes confined to local optima in stochastic decoding and thus fails to exhaustively explore potential draft outputs to maximize acceptance rates.

Fig.4 illustrates the typical flow of the MSS verification process. As the method encounters a branching point within the search tree, it sequentially evaluates each child node to determine whether it satisfies the acceptance criteria. Upon accepting a child node, the algorithm proceeds along that branch, effectively abandoning the evaluation of other child nodes at that particular level. In other words, once a branching decision is made, the algorithm is committed to a single path, precluding any further exploration of alternative branches that might offer broader token acceptance ranges. This inherent limitation results in an exploration process that may get stuck at a local optimum, as the algorithm tends to overlook branches that could yield more optimal results across the token tree.

This limitation becomes particularly problematic in stochastic decoding, where the need for broader exploration is essential to improving overall performance. The tendency to converge prematurely on a limited range of solutions significantly hinders the algorithm’s ability to maximize its acceptance rates, thus undermining the potential benefits of the multi-draft sampling strategy.

V. DESIGN

A. System Overview

In this section, we elaborate on the system design details, as illustrated in Figure 5. SuperSpec implements end-to-end optimization for the speculative decoding workflow from drafting to verification. To address critical challenges in speculative decoding systems, such as verification latency, local optima in stochastic sampling, and dynamic adjustment of prediction rounds across multiple drafters, SuperSpec proposes three pivotal components: the *Efficient Batch Verifier*, the *Global Optimal Sampler*, and the *Dynamic Adaptive Multi-Drafter*.

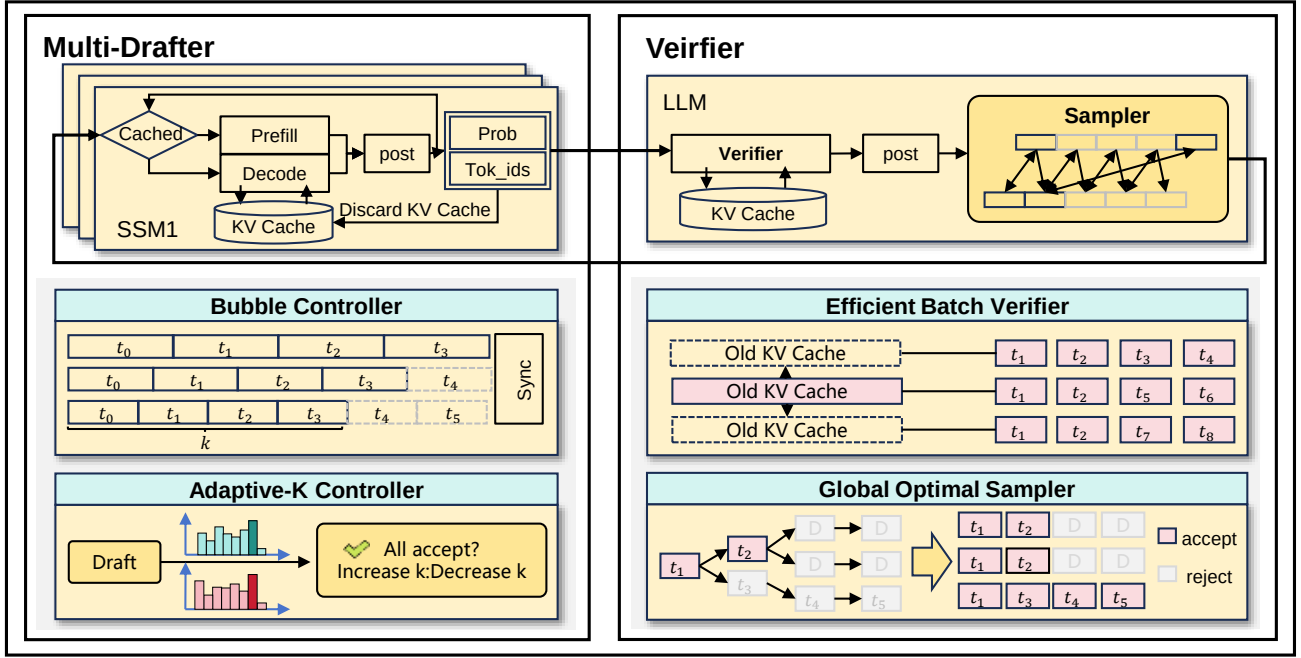


Fig. 5: SuperSpec Overview.

B. Efficient Batch Verifier

In order to solve the problem of long verification time of tree-based decoding for longer sequences to be verified, SuperSpec designed the batch-based decoding method and proposed the efficient batch verifier based on it. The process of this new decoding algorithm, which is different from tree-based decoding, is illustrated in Efficient Batch Verifier in Fig.5.

Unlike the traditional tree-based decoding method, the batch-based decoding approach does not flatten the batch-verified sequences in the verification stage, but packs them into a batch and sends them to LLM for verification. The *batch_size* of the validation is related to the channel to get the draft tokens. When we use different small speculative models (SSMs), the *batch_size* of the validation is the number of SSMs; when we use enhanced retrieval, string matching, and SSM autoregressive generation for the same request, the *batch_size* of the validation is 3. Batch-Based Decoding The main advantage of the batch-based decoding approach is that it can fully utilise the parallel computing power of the GPU. By validating multiple shorter sequences at the same time, the batch-based verifier reduces the validation latency compared to processing only one longer sequence at a time.

We define a complete draft, verify and sample as a round in speculative decoding. We note that while the batch-based approach achieves latency gains, the explicit memory of the verification phase stores *batch_size* copies. For a given request, each SSM stores its own KV Cache, and the LLM stores its own KV Cache, both of which are tightly bound to their respective models and cannot be reused with each other. When decoding the i th round, a naive approach to ensure the batch-

based decoding to work properly is to copy all KV Cache of $[0, i - 1]$ rounds, which will consume too much memory when the sequence length is long, resulting in out of memory error.

To alleviate the problem of high memory usage in batch-based decoding, efficient batch verifier proposes a layer-by-layer replication strategy for KV Cache, which reduces the peak memory usage to $1/L$, where L is the number of layers in the model. The verify phase of the model needs to go through each layer of forward, and the KV Cache is stored in layers. Compared with directly copying the KV Cache of all layers, batch-based decoding chooses a layer-by-layer copying strategy, which copies the KV Cache of the current layer when the model is decoded to each layer, thus ensuring the decoding process is carried out normally, and at the same time guaranteeing a lower peak memory usage.

In many different datasets[4, 9, 31, 35], the draft tokens obtained from multiple draft models may vary greatly, resulting in a long sequence to be verified. batch-based decoding reduces the latency of the verification phase while minimising the redundancy of memory usage by designing the memory management strategy of replicating the KV Cache on a layer-by-layer basis.

C. Global Optimal Sampler

Traditional tree-based decoding algorithms use a rejection sampling algorithm for stochastic decoding[6], a significant drawback of which is that sampling does not ensure that the optimal solution in the token tree is selected.

As shown in the Global Optimal Sampler in Fig.5, tree-based sampling randomly selects a child node of the current node for subsequent sampling when a node is sampled. The

sampling algorithm will exit the loop if the currently selected node token is rejected by the LLM and no other node exists in the node pair column. The left side of the Global Optimal Sampler in Fig.5 shows the process of tree-based sampling. When decoding to the t_2 node, because t_2 is rejected, and the node pair column is empty at this time, it is not possible to continue to down-sample, and only the locally optimal solution of t_1t_2 is taken, and does not take the potentially longer to be verified sequences $t_1t_3t_4t_5$. The batch-based sampling algorithm, on the other hand, is based on batch-based decoding, and makes a sampling judgement for each draft request in the batch in the verification phase, and ultimately retains the draft sequence with the longest received length.

The main advantage of batch-based decoding is that it can efficiently explore all draft sequences at the same time and select the longest and optimal acceptance sequence. batch-based sampler is significantly better than tree-based sampler using MSS sampling mechanism. This is because the batch-based sampler ensures that the number of accepted tokens in each round of speculative decoding is always greater than or equal to the number of accepted tokens in the tree-based sampler. Since the batch-based sampler does not change the sampling rules for random decoding, but effectively utilises the parallel computing power to comprehensively search all draft sequences instead of opportunistically selecting locally optimal sampling paths, the batch-based sampler does not affect the quality of the generated text, nor does it increase the sampling latency. Thus, the batch-based sampler increases the acceptance rate, reduces the number of speculative decoding rounds, and decreases the end-to-end latency.

D. Dynamic Adaptive Multi-Drafter

The multi-drafter system allows SuperSpec to dynamically adjust the number of tokens predicted in each round of speculative decoding (denoted as K). By monitoring the idleness of each drafter during the generation process, the system adapts the K for each drafter, ensuring optimal performance. As shown in Bubble Controller of Fig.5, this adaptive mechanism alleviates the "bubble phenomenon" that can occur when drafters with different speeds are forced to use the same K , leading to inefficiency.

In speculative sampling, K is predicted in each speculative decoding round is critical for the system performance. If K is set too small, the system cannot fully take advantage of the acceleration effects offered by speculative sampling. In contrast, if K is set too large, the acceptance rate drops, leading to an increase in latency. In multi-drafter systems, the main challenge lies in determining the optimal K for each drafter to ensure that the system operates with optimal performance. To address this, SuperSpec introduces Algorithm 1 to adjust K of multiple drafters, in order to mitigate the bubble phenomenon and resolve the optimal problem K .

1) *Alleviate the Bubble Phenomenon*: In a multi-drafter speculative sampling system, the time required for multiple drafters to generate a token can vary significantly. A major issue arises when all drafters use the same K , which causes

Algorithm 1 Dynamic K Adjustment for Multi-Drafter

Input Drafters = $\{D_1, D_2, \dots, D_n\}$, latency profiles $\{T_1, T_2, \dots, T_n\}$
Output Optimized K-values $\{K_1, K_2, \dots, K_n\}$
1: Initialize $K = \{K_1, K_2, \dots, K_n\}$
2: **for** each round r of speculative decoding **do**
3: Compute temporal product set $P \leftarrow \{T_i \times K_i \mid 1 \leq i \leq n\}$
4: Find maximum latency $M \leftarrow \max(P)$
5: **for** each drafter D_i **do**
6: Calculate idle time $I_i \leftarrow M - (T_i \times K_i)$
7: Compute optimal K-value $K'_i \leftarrow I_i / T_i$
8: Update $K_i \leftarrow \lfloor K_i + K'_i \rfloor$
9: $K_i \leftarrow \max(1, \min(K_i, K_{\max}))$
10: **end for**
11: **end for**

faster drafters to become idle while waiting for slower models to generate the K tokens. This situation is referred to as the bubble phenomenon. To alleviate this issue, SuperSpec introduces an adaptive algorithm that dynamically adjusts the K for each drafter based on its level of idleness during each speculative decoding round. This adaptive approach ensures that, after several rounds of speculative decoding, each drafter's K converges to an optimal value. As a result, all drafters are able to complete their speculations simultaneously, effectively eliminating the bubble phenomenon. Consequently, each drafter can fully utilize its draft time to perform more speculative steps, thereby enhancing the overall performance of the speculative sampling system.

2) *Adaptive adjustment of K*: The setting of the K directly affects the acceptance rate of speculative reasoning, which in turn impacts the overall performance of the system. SuperSpec adopts a heuristic method for dynamically adjusting the K in an attempt to find the optimal value. In each round of speculative decoding, if a drafter's speculative sequence is fully accepted, the K for all drafters is increased by one. Conversely, if all drafter speculative sequences are fully rejected, the K for all drafters is decreased by one. The adjustment of the K is based on the acceptance status of the draft results, rather than setting a fixed K . The advantage of this approach is that the algorithm can automatically adjust the K to a suitable level based on the context and the current system performance, ensuring the stability of the acceptance rate.

VI. EVALUATION

A. Setup

We implemented all features of SuperSpec using Python and PyTorch to enable inference acceleration for the entire end-to-end process of speculative decoding, with support for multiple draft models for parallel inference and verifiers for efficient parallel verification.

1) *Model Configuration*: To demonstrate the universality of SuperSpec, we evaluate its performance on Qwen2.5[34]

and OPT[36] series model. Specifically, we select OPT-66B and Qwen2.5-72B-Instruct as LLM, and OPT-125M, OPT-350M, OPT-1.3B, as well as Qwen2.5-0.5B-Instruct, Qwen2.5-1.5B-Instruct, and Qwen2.5-3B-Instruct as the corresponding small models for draft.

2) **Workload:** We evaluated the performance of SuperSpec using a diverse set of workloads: Alpaca[4], GSM8K[5], XSUM[37], and HumanEval[35]. The workloads comprehensively cover critical language processing domains including cross-domain instruction adherence, mathematical reasoning, code synthesis, multi-turn dialogue coherence and extreme summarization. This systematic coverage ensures rigorous evaluation of SuperSpec’s adaptability across different domains.

3) **Baselines:** We use autoregressive decoding as well as different configurations of SpecInfer[6] as a baseline to fully demonstrate the enhancement of SuperSpec over the widely used tree-based verification mechanism. For a comprehensive comparison, we construct token trees for different numbers of miniatures and analyze the results under different lookahead.

B. Acceptance Rate Results

Table.I compares the acceptance rates of SuperSpec and SpecInfer with tree-based validation on five benchmark datasets under ($k = 2$ to $k = 6$) different speculation lengths (temperature set to 0.8, LLM model OPT-66B). Across all datasets and for different values of speculative length, SuperSpec consistently outperforms SpecInfer using the tree-based validation approach, achieving up to a 79.2% improvement in acceptance rate from 48% to 86%. This improvement is due to the fact that SpecInfer’s tree sampling chance is too large when the speculative length is small, reflecting the superiority of the batch sample global optimal solution in SuperSpec. When the speculative length range is varied from 2 to 6, the average acceptance rate improvement of SuperSpec over the five datasets is 30.2%, 11.6%, 6.4%, 17.2% and 20.7%, respectively. These results highlight SuperSpec’s superior token utilisation efficiency across a wide range of generation tasks, which further reduces verify latency and improves end-to-end responsiveness.

C. End-to-End Latency Results

To comprehensively evaluate SuperSpec’s performance, we conducted rigorous latency measurements across three benchmark datasets (Alpaca, GSM8K, XSUM) under varying lookahead lengths ($k=2$ to $k=6$) and temperature settings ($T=0.0$ and $T=0.8$). The results in Fig.6 show that SuperSpec is much improved compared to the tree-based validation SpecInfer, with average speedups ranging from 7.12% to 62.06% in the Qwen2.5-72B and OPT-66B model families. On the Qwen2.5-72B model, the mean median speedup improvement for the Alpaca, GSM8K and XSUM datasets are 61.64%, 51.39% and 49.87% respectively. On the OPT-66B model, the mean median speedup for the three datasets is 33.21%, 17.95%, and 22.98%, respectively, using random sampling with temperature=0.8.

TABLE I: Comparison of SuperSpec and SpecInfer Acceptance Rates at OPT-66B, $T=0.8$

	Dataset	Speculative length				
		2	3	4	5	6
SpecInfer	Alpaca	48%	69%	60%	52%	49%
	GSM8K	80%	69%	61%	60%	51%
	HumanEval	82%	81%	66%	57%	54%
	MTBench	75%	66%	61%	52%	57%
	XSUM	72%	69%	61%	56%	47%
SuperSpec	Alpaca	86%	80%	68%	65%	58%
	GSM8K	85%	77%	74%	64%	58%
	HumanEval	83%	86%	74%	61%	57%
	MTBench	81%	74%	78%	67%	62%
	XSUM	85%	80%	73%	66%	62%

We note that SuperSpec achieves different speedup gains on datasets with different task types. For Qwen2.5-72B at $k=6$, SuperSpec’s median speedup on the Alpaca dataset is 1.73 times that of autoregressive inference, which is a 96.6% increase in speedup gain compared to SpecInfer, and for the GSM8K dataset this speedup gain is 75.15% while for the XSUM dataset this speedup gain is 53.8% for the GSM8K dataset and 53.8% for the XSUM dataset. This is due to the fact that SuperSpec’s globally optimal sampler is able to fetch the maximum number of accepted sequences to be verified during the verification phase, and this sampler avoids premature path truncation on different tasks in the tree-based verifier SpecInfer. As shown in Table.I, SuperSpec’s bulk sampling increases the acceptance rate of pending validation sequences by 8%-38% per round over the length of Alpaca’s multistep speculation, directly amortizing the validation overhead through a larger number of accepted tokens.

Based on the experimental results in Fig.1 we note that the speculative length Lookahead(K) significantly affects the speedup ratio of SuperSpec, and different tasks have different optimal values of K. This empirical finding highlights a key challenge in speculative decoding: static K configurations may not yield optimal performance across different workloads. The observed variability underscores the importance and efficacy of adaptive mechanisms in speculative decoding systems. For SuperSpec, this task-dependent optimality of K validates the design of components such as the Adaptive-K Controller in Multi-Drafter, which dynamically adapts speculative lengths based on real-time metrics. This adaptability is critical to the trade-off between aggressive speculation and acceptance rates, enabling SuperSpec to maintain high efficiency and significantly reduce end-to-end latency across a wide range of benchmarks and operating conditions.

Sampling temperature affects the randomness of the samples, and SuperSpec accelerates more effectively at higher temperatures and greater randomness. For the OPT-66B model, the average speedup ratio enhancement over SpecInfer for different speculative lengths is 9.41%, 7.12% and 8.14% for $T=0$ on the Alpaca, GSM8K and XSUM datasets, respectively, while the average speedup ratio enhancement at $T=0.8$ is 32.51%, 13.74% and 22.24%. This stems from the ability of SuperSpec’s global sampler to explore non-greedy paths:

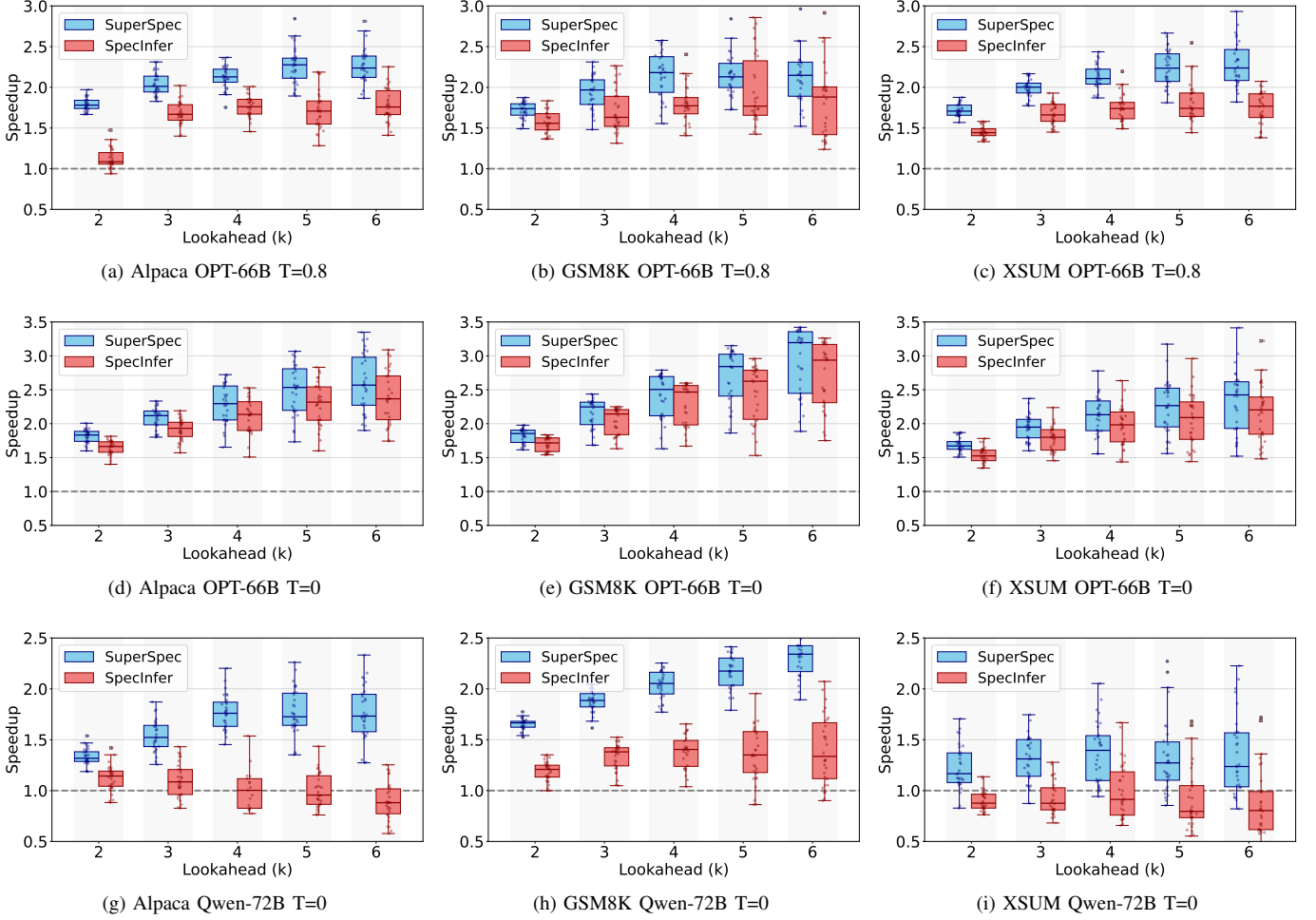


Fig. 6: SuperSpec End-to-End Latency Speedup on OPT-66B and Qwen2.5-72B Models

whereas tree sampling-based SpecInfer discards divergent draft tokens prematurely, SuperSpec retains these drafts for parallel evaluation, resulting in an increase in average acceptance rate of valid tokens on the Alpaca, GSM8K, and XSUM datasets by 16.17 %, 7.59% and 12.07% respectively.

VII. CONCLUSION

We have presented SuperSpec, a novel speculative decoding system that alleviates the overlooked bottlenecks in verification latency, suboptimal sampling paths and inefficient multi-drafter coordination. SuperSpec introduces three key innovations: (1) The *Efficient Batch Verifier* for GPU-friendly parallel verification with layer-wise KV cache replication. (2) The *Global Optimal Sampler* for exhaustive token sequence exploration. (3) The *Dynamic Adaptive Multi-Drafter* for bubble-free resource allocation. SuperSpec achieves significant improvements in both decoding efficiency and acceptance rates. Extensive experiments on different datasets of Qwen2.5-72B and OPT-66b show that SuperSpec outperforms the state-of-the-art tree-based verification system SpecInfer. SuperSpec improves candidate token acceptance rate by 6.4%-30.2%

compared to the existing methods, and improves the end-to-end average inference speedup ratio compared to SpecInfer by 7.12%-62.06%. These improvements are achieved without compromising the quality of text generation, making SuperSpec a powerful solution for accelerating LLM inference in real-world scenarios.

REFERENCES

- [1] L. Mai and J. Carson-Berndsen, “Real-time textless dialogue generation,” *arXiv preprint arXiv:2501.04877*, 2025.
- [2] M. Izadi, J. Katzy, T. Van Dam, M. Otten, R. M. Popescu, and A. Van Deursen, “Language models for code completion: A practical evaluation,” in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, 2024, pp. 1–13.
- [3] S.-T. Pi, C.-P. Hsieh, Q. Liu, and Y. Zhu, “Universal model in online customer service,” in *Companion Proceedings of the ACM Web Conference 2023*, 2023, pp. 878–885.

- [4] R. Taori, I. Gulrajani, T. Zhang, Y. Dubois, X. Li, and T. B. Hashimoto, "Stanford alpaca: An instruction-following llama model," https://github.com/tatsu-lab/stanford_alpaca, 2023.
- [5] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, and J. Schulman, "Training verifiers to solve math word problems," *arXiv preprint arXiv:2110.14168*, 2021.
- [6] X. Miao, G. Oliaro, Z. Zhang, X. Cheng, Z. Wang, and X. Shi, "Specinfer: Accelerating large language model serving with tree-based speculative inference and verification," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, 2024, pp. 932–949.
- [7] H. Xia, Y. Li, J. Zhang, C. Du, and W. Li, "Swift: On-the-fly self-speculative decoding for llm inference acceleration," *arXiv preprint arXiv:2410.06916*, 2024.
- [8] T. Cai, Y. Li, Z. Geng, H. Peng, J. D. Lee, and T. Dao, "Medusa: Simple llm inference acceleration framework with multiple decoding heads," *arXiv preprint arXiv:2401.10774*, 2024.
- [9] A. See, P. J. Liu, and C. D. Manning, "Get to the point: Summarization with pointer-generator networks," in *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Vancouver, Canada: Association for Computational Linguistics, Jul. 2017, pp. 1073–1083. [Online]. Available: <https://www.aclweb.org/anthology/P17-1099>
- [10] M. Shi, F. Liu, S. Wang, S. Liao, S. Radhakrishnan, and Y. Yacoob, "Eagle2: Exploring the design space for multimodal llms with mixture of encoders," *arXiv preprint arXiv:2408.15998*, 2024.
- [11] Y. Li, F. Wei, C. Zhang, and H. Zhang, "Eagle: Speculative sampling requires rethinking feature uncertainty," *arXiv preprint arXiv:2401.15077*, 2024.
- [12] A. Chio, D. Jiang, P. Gupta, and N. Venkatasubramanian, "Smartspec: Customizable smart space datasets via event-driven simulations," in *2022 IEEE International Conference on Pervasive Computing and Communications (PerCom)*. IEEE, 2022, pp. 152–162.
- [13] Z. Chen, A. May, R. Svirschevski, Y.-H. Huang, M. Ryabinin, and Z. Jia, "Sequoia: Scalable and robust speculative decoding," *Advances in Neural Information Processing Systems*, vol. 37, pp. 129 531–129 563, 2024.
- [14] J. Wang, Y. Su, J. Li, Q. Xia, Z. Ye, and X. Duan, "Opt-tree: Speculative decoding with adaptive draft tree structure," *Transactions of the Association for Computational Linguistics*, vol. 13, pp. 188–199, 2025.
- [15] X. Luo, Y. Wang, Q. Zhu, Z. Zhang, X. Zhang, and W. Che, "Turning trash into treasure: Accelerating inference of large language models with token recycling," *arXiv preprint arXiv:2408.08696*, 2024.
- [16] W. Zhao, Y. Huang, X. Han, W. Xu, C. Xiao, and X. Zhang, "Ouroboros: Generating longer drafts phrase by phrase for faster speculative decoding," *arXiv preprint arXiv:2402.13720*, 2024.
- [17] Y. Fu, P. Bailis, I. Stoica, and H. Zhang, "Break the sequential dependency of llm inference using lookahead decoding," *arXiv preprint arXiv:2402.02057*, 2024.
- [18] N. Yang, T. Ge, L. Wang, B. Jiao, and F. Wei, "Inference with reference: Lossless acceleration of large language models," *arXiv preprint arXiv:2304.04487*, 2023.
- [19] Z. He, Z. Zhong, T. Cai, J. D. Lee, and D. He, "Rest: Retrieval-based speculative decoding," *arXiv preprint arXiv:2311.08252*, 2023.
- [20] F. Liu, Y. Tang, Z. Liu, Y. Ni, and Y. Wang, "Kangaroo: Lossless self-speculative decoding via double early exiting," *arXiv preprint arXiv:2404.18911*, 2024.
- [21] T. Liu, Y. Li, Q. Lv, K. Liu, J. Zhu, and W. Hu, "Pearl: Parallel speculative decoding with adaptive draft length," in *The Thirteenth International Conference on Learning Representations*.
- [22] B. Butler, S. Yu, A. Mazaheri, and A. Jannesari, "Pipeinfer: Accelerating llm inference using asynchronous pipelined speculation," in *SC24: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 2024, pp. 1–19.
- [23] S. Wang, H. Yang, X. Wang, T. Liu, P. Wang *et al.*, "Minions: Accelerating large language model inference with aggregated speculative execution," *arXiv preprint arXiv:2402.15678*, 2024.
- [24] L. Gao, J. Liu, H. Xu, and L. Huang, "Collaborative speculative inference for efficient llm inference serving," *arXiv preprint arXiv:2503.10325*, 2025.
- [25] N. Bhendawade, I. Belousova, Q. Fu, H. Mason, M. Rastegari, and M. Najibi, "Speculative streaming: Fast llm inference without auxiliary models," *arXiv preprint arXiv:2402.11131*, 2024.
- [26] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper, "Accelerating large language model decoding with speculative sampling," *arXiv preprint arXiv:2302.01318*, 2023.
- [27] Y. Leviathan, M. Kalman, and Y. Matias, "Fast inference from transformers via speculative decoding," in *International Conference on Machine Learning*. PMLR, 2023, pp. 19 274–19 286.
- [28] M. Stern, N. Shazeer, and J. Uszkoreit, "Blockwise parallel decoding for deep autoregressive models," *Advances in Neural Information Processing Systems*, vol. 31, 2018.
- [29] H. M. Chen, W. Luk, K. F. C. Yiu, R. Li, K. Mishchenko, and H. Fan, "Hardware-aware parallel prompt decoding for memory-efficient acceleration of llm inference," *arXiv preprint arXiv:2405.18628*, 2024.
- [30] Z. Yuan, Y. Shang, Y. Zhou, Z. Dong, Z. Zhou, and C. Xue, "Llm inference unveiled: Survey and roofline model insights," *arXiv preprint arXiv:2402.16363*, 2024.
- [31] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, and I. Stoica, "Judging llm-as-a-judge with mt-bench and chatbot arena," 2023.
- [32] J. Liu, Y. Fu, R. Xie, R. Xie, X. Sun, and X. Li, "Phd: A chatgpt-prompted visual hallucination evaluation

dataset,” *arXiv preprint arXiv:2403.11116*, 2024.

- [33] J. Berant, A. Chou, R. Frostig, and P. Liang, “Semantic parsing on freebase from question-answer pairs,” in *Proceedings of the 2013 conference on empirical methods in natural language processing*, 2013, pp. 1533–1544.
- [34] Q. Team, “Qwen2.5: A party of foundation models,” September 2024. [Online]. Available: <https://qwenlm.github.io/blog/qwen2.5/>
- [35] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, and W. Zaremba, “Evaluating large language models trained on code,” 2021.
- [36] S. Zhang, S. Roller, N. Goyal, M. Artetxe, M. Chen, and L. Zettlemoyer, “Opt: Open pre-trained transformer language models,” 2022.
- [37] S. Narayan, S. B. Cohen, and M. Lapata, “Don’t give me the details, just the summary! topic-aware convolutional neural networks for extreme summarization,” *ArXiv*, vol. abs/1808.08745, 2018.